SESIONES EN PHP

1. ¿QUÉ ES UNA SESIÓN?

Cuando un usuario visita una página web, el navegador envía una petición HTTP al servidor. El servidor procesa la petición y devuelve una respuesta. Si el usuario solicita otra página, se realiza una nueva petición HTTP.

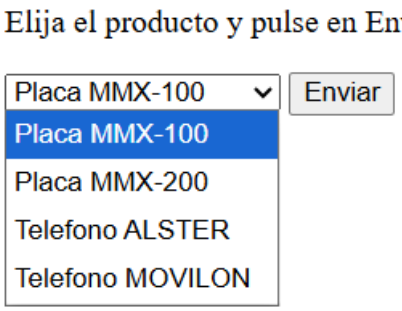
El problema es que HTTP es un protocolo sin estado: por sí mismo, no recuerda qué ha ocurrido en las peticiones anteriores. Por ejemplo, si un usuario selecciona un producto y después accede a otra página, PHP no conserva automáticamente el valor de las variables normales que se utilizaron en la primera petición.

Para solucionar este problema, PHP dispone de las sesiones. Una sesión permite almacenar información asociada a un usuario y recuperarla durante sus siguientes peticiones a la aplicación.

Esta información se almacena normalmente en el servidor y se gestiona mediante el array superglobal **$_SESSION**.

Ejemplo: una tienda online

Imagina que un usuario selecciona una placa de 700 euros. Después, selecciona un teléfono de 850 euros. La aplicación debe recordar el primer producto para calcular el importe total.



Primera petición: seleccionar una placa

- Navegador
  - Placa MMX-100 — 700 €

Envía el formulario

- Servidor PHP
  - $_SESSION["Total"] = 700

Segunda petición: seleccionar un teléfono

- Navegador
  - Teléfono ALSTER — 850 €

Envía otro formulario

- Servidor PHP
  - $_SESSION["Total"] += 850
  - Total: 1.550 €

La sesión permite recuperar el importe anterior y actualizarlo. Sin embargo, hay que tener en cuenta que los datos no se conservan por utilizar una variable PHP normal, sino porque se guardan en **$_SESSION.**

2. ¿CÓMO IDENTIFICA PHP A CADA USUARIO?

El servidor debe saber qué información de sesión corresponde a cada navegador. Para ello, PHP utiliza normalmente una cookie con un identificador de sesión, cuyo nombre predeterminado es **PHPSESSID**.

El funcionamiento es el siguiente:

1. El navegador solicita una página PHP.
2. PHP inicia una sesión o recupera una ya existente.
3. El servidor envía al navegador una cookie con el identificador de sesión cuando corresponde.
4. En las siguientes peticiones, el navegador envía esa cookie.
5. PHP utiliza el identificador para recuperar los datos asociados a esa sesión.

Ejemplo con dos usuarios

- Usuario A
  - Identificador ficticio:
    - abc123
  - Total, guardado
    - **700 €**
- Usuario B
  - Identificador ficticio:
    - xyz789
  - Total, guardado
    - **1.550 €**
- Servidor PHP
  - Mantiene los datos de cada sesión por separado.

Los identificadores anteriores son ejemplos, no identificadores reales. El objetivo es comprender que dos usuarios pueden utilizar la misma aplicación y tener valores diferentes en **$_SESSION["Total"].**

La cookie no almacena directamente los 700 o los 1.550 euros: normalmente contiene el identificador que permite al servidor localizar los datos de la sesión.

3. EL ARRAY SUPERGLOBAL $_SESSION

PHP proporciona el array superglobal **$_SESSION** para almacenar información que debe mantenerse entre diferentes peticiones del mismo usuario.

Se denomina superglobal porque puede utilizarse desde cualquier ámbito del programa sin necesidad de declararlo mediante global.

Para trabajar con sesiones hay que ejecutar primero:

```
<?php
        session_start();
?>
```

Esta función debe ejecutarse antes de enviar contenido HTML al navegador. Si la sesión ya existe y el identificador es válido, PHP recupera los datos correspondientes. Si no existe una sesión identificable, puede iniciar una nueva.

A. GUARDAR INFORMACIÓN EN LA SESIÓN

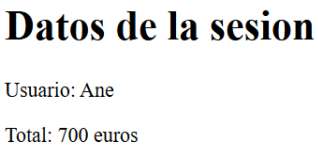
Crea un archivo llamado **ejemplo_sesion.php** y escribe el siguiente código:

```
<?php
        session_start();
        // Guardamos informacion en la sesion
        $_SESSION["usuario"] = "Ane";
        $_SESSION["Total"] = 700;
?>
<!DOCTYPE html>
        <html lang="es">
                <head>
                        <meta charset="UTF-8">
                        <title>Ejemplo de sesion</title>
                </head>
                        <body>
                        <h1>Datos de la sesion</h1>
                        <p>Usuario: <?php echo $_SESSION["usuario"]; ?></p>
                        <p>Total: <?php echo $_SESSION["Total"]; ?> euros</p>
                </body>
        </html>
```

Guarda el archivo en la carpeta de tu servidor web, por ejemplo, C:\xampp\htdocs\DSER\Public\UD03\Session\ejemplo_sesion.php, inicia Apache y accede a:

http://localhost:8000/UD03/Session/ejemplo_sesion.php

El navegador mostrará:



Vamos a analizar las instrucciones:

- **session_start()** inicia o recupera la sesión.
- **$_SESSION["usuario"] = "Ane"** crea una entrada llamada usuario y almacena el texto Ane.
- **$_SESSION["Total"] = 700** crea una entrada llamada Total y almacena el número 700.
- echo **$_SESSION["Total"]** recupera y muestra el importe almacenado.

B. MODIFICAR LA INFORMACIÓN DE LA SESIÓN

Ahora vamos a crear un segundo archivo, **modificar_sesion.php**, que recuperará el total anterior y le sumará 850 euros.

```
<?php
        session_start();
        // Comprobamos si existe el total
        if (!isset($_SESSION["Total"])) {
            $_SESSION["Total"] = 0;
        }
        // Sumamos 850 euros al total
        $_SESSION["Total"] += 850;
?>
```

```
<!DOCTYPE html>
    <html lang="es">
        <head>
            <meta charset="UTF-8">
            <title>Modificar sesion</title>
        </head>
        <body>
            <h1>Actualizacion del total</h1>
            <p>Total actual:
                <?php echo $_SESSION["Total"]; ?> euros
            </p>
            <p>
                <a href="modificar_sesion.php">Sumar otros 850 euros</a>
            </p>
        </body>
    </html>
```

Importante: ejecuta primero **ejemplo_sesion.php** y después **modificar_sesion.php** en el mismo navegador. Ambos archivos deben pertenecer a la misma aplicación y utilizar el mismo identificador de sesión.

Si el total inicial era **700 euros**, la primera ejecución de **modificar_sesion.php** mostrará **1.550 euros**. Si vuelves a pulsar el enlace, sumará otros **850 euros** y mostrará **2.400 euros.**

Esto demuestra que la sesión permite conservar y modificar información entre peticiones HTTP diferentes.

4. EJEMPLO COMPLETO: CALCULAR EL TOTAL DE UNA COMPRA

Ahora pasaremos a desarrollar un ejemplo de una tienda donde el usuario seleccionará productos de una lista desplegable y PHP calculará el importe acumulado mediante una sesión.

La aplicación tendrá cuatro partes:

1. Mostrar el formulario cuando todavía no se ha seleccionado ningún producto.
2. Recibir el producto seleccionado mediante **$_POST**.
3. Sumar su precio al total almacenado en **$_SESSION.**
4. Mostrar el importe y permitir volver al formulario.

A. CÓDIGO COMPLETO DE SESIONES.PHP

Copia el siguiente código en un archivo llamado **sesiones.php.**

```
<?php
        // Iniciamos o recuperamos la sesion
        session_start();
        // Inicializamos el total la primera vez
        if (!isset($_SESSION["Total"])) {
            $_SESSION["Total"] = 0;
        }
?>
<!DOCTYPE html>
<html lang="es">
        <head>
            <meta charset="UTF-8">
            <title>Sesiones</title>
        </head>
```

```
<body>
        <?php
        // Comprobamos si se ha enviado el formulario
        if (!isset($_POST["botEnviar"])) {
        ?>
            <h1>Seleccion de productos</h1>
            <p>Elija el producto y pulse en Enviar:</p>
            <form method="post"
                action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]); ?>">
                <select name="Productos" size="1">
                    <option value="Placa MMX-100">Placa MMX-100</option>
                    <option value="Placa MMX-200">Placa MMX-200</option>
                    <option value="Telefono ALSTER">Telefono ALSTER</option>
                    <option value="Telefono MOVILON">Telefono MOVILON</option>
                </select>
                <input type="submit" name="botEnviar" value="Enviar">
            </form>
        <?php
        } else {
            // Recuperamos el producto seleccionado
            $producto = $_POST["Productos"];
            // Calculamos el importe
            switch ($producto) {
                case "Placa MMX-100":
                    $_SESSION["Total"] += 700;
                    break;
                case "Placa MMX-200":
                    $_SESSION["Total"] += 1400;
                    break;
                case "Telefono ALSTER":
                    $_SESSION["Total"] += 850;
                    break;
                case "Telefono MOVILON":
                    $_SESSION["Total"] += 623;
                    break;
            }
```

```
            // Mostramos el total acumulado
            echo "<h2>Total a facturar: "
                . $_SESSION["Total"] . " euros</h2>";
            // Enlace para volver al formulario
            echo '<p><a href="' . htmlspecialchars($_SERVER["PHP_SELF"])
                . '">Volver</a></p>';
        }
        ?>
    </body>
</html>
```

B. ¿QUÉ OCURRE CUANDO EJECUTAMOS EL PROGRAMA?

La primera vez que accedemos a **sesiones.php**, todavía no existe el botón **botEnviar** en los datos recibidos. Por eso se ejecuta el primer bloque if y se muestra el formulario.

Cuando seleccionamos un producto y pulsamos «Enviar», el navegador envía los datos al servidor mediante el método **POST**. En esa nueva petición, **isset($_POST["botEnviar"])** devuelve true y se ejecuta el bloque else.

El código recupera el producto, identifica su precio mediante switch y actualiza el total de la sesión.

C. COMPRUEBA EL RESULTADO

Simulación de las operaciones de compra.

D. ¿POR QUÉ UTILIZAMOS ISSET()?

La instrucción:

```
if (!isset($_SESSION["Total"])) {
    $_SESSION["Total"] = 0;
}
```

comprueba si la variable Total existe en la sesión.

- Si no existe, la inicializamos a cero.
- Si ya existe, mantenemos su valor anterior.

Si no comprobásemos su existencia, podríamos intentar sumar un precio a una variable que todavía no se ha inicializado.

También es importante no reiniciar el total a cero en cada petición. Si escribiéramos directamente **$_SESSION["Total"] = 0** cada vez que se ejecuta la página, perderíamos el importe acumulado antes de sumar el nuevo producto.

5. FINALIZAR UNA SESIÓN

Durante una aplicación web, puede ser necesario finalizar una sesión, por ejemplo, cuando un usuario pulsa «Desconectar» o cuando termina una operación.

Hay que distinguir entre cerrar el navegador y destruir una sesión.

Cerrar el navegador no garantiza que los datos se eliminen inmediatamente del servidor. PHP puede conservar los datos hasta que se ejecute el proceso de limpieza de sesiones caducadas. Para terminar una sesión explícitamente, disponemos de la función **session_destroy().**

A. UTILIZAR SESSION_DESTROY()

```
<?php
        session_start();
        $_SESSION = [];
        session_destroy();
        echo "Sesion finalizada";
?>
```

- **session_start()** permite acceder a la sesión actual.
- **$_SESSION = [];** vacía las variables de sesión de la petición actual.
- **session_destroy()** destruye los datos asociados a la sesión en el servidor.

La función **session_destroy()** no elimina automáticamente la cookie del navegador ni vacía por sí sola el array **$_SESSION** de la petición actual. Por eso, en el ejemplo, vaciamos el array antes de destruir la sesión.

B. AÑADIR UN BOTÓN DE DESCONEXIÓN

En una aplicación, podemos utilizar un formulario para enviar al servidor la petición de desconexión.

```
<form method="post" action="sesiones.php">
    <input type="submit" name="botDesconectar" value="Desconectar">
</form>
```

El atributo **name="botDesconectar"** permite identificar la petición en PHP:

```
if (isset($_POST["botDesconectar"])) {
    $_SESSION = [];
    session_destroy();
    echo "<p>Sesion finalizada</p>";
}
```

Cuando se pulsa el botón, el navegador envía una petición POST. PHP comprueba si existe el campo **botDesconectar** y, si existe, ejecuta el código de desconexión.

C. MOSTRAR EL TOTAL ANTES DE DESTRUIR LA SESIÓN

En el ejercicio de la tienda, puede que queramos que el importe final siga apareciendo después de desconectar. Para hacerlo, tenemos que guardar el valor en una variable normal antes de destruir la sesión.

```
$totalFinal = $_SESSION["Total"] ?? 0;
$_SESSION = [];
session_destroy();
echo "<p>Sesion finalizada</p>";
echo "<p>Total final: " . $totalFinal . " euros</p>";
```

La variable **$totalFinal** conserva el importe durante la ejecución de esa petición, aunque los datos de la sesión se hayan destruido. Sin embargo, si se realiza otra petición, esa variable normal ya no conserva automáticamente el valor.

Este detalle es importante: destruir una sesión elimina los datos de sesión, pero no elimina las variables PHP normales que ya existen durante la petición actual.

6. ¿CUÁNDO TERMINA UNA SESIÓN?

Una sesión puede finalizar de distintas formas:

1. Mediante código PHP, utilizando **session_destroy().**
2. Por caducidad, cuando PHP elimina los datos de una sesión que ya no se considera activa, según la configuración del servidor.
3. Al cerrar el navegador, puede desaparecer la cookie de sesión si está configurada como una cookie temporal, pero esto no implica que los datos se eliminen inmediatamente del servidor.

La duración de la cookie y el tiempo durante el que los datos permanecen en el servidor son conceptos diferentes.

Por ejemplo, una cookie temporal puede desaparecer al cerrar el navegador, mientras que los datos correspondientes a esa sesión pueden permanecer almacenados en el servidor hasta que se eliminen.

Ejemplo para comprobarlo

Crea un archivo llamado **comprobar_sesion.php:**

```
<?php
        session_start();
        if (!isset($_SESSION["visitas"])) {
            $_SESSION["visitas"] = 1;
        } else {
            $_SESSION["visitas"]++;
        }
?>
<!DOCTYPE html>
        <html lang="es">
                <head>
                    <meta charset="UTF-8">
                    <title>Contador de visitas</title>
                </head>
                <body>
                    <h1>Contador de visitas</h1>
                    <p>Numero de peticiones:
                        <?php echo $_SESSION["visitas"]; ?>
                    </p>
                    <p>
                        <a href="comprobar_sesion.php">Volver a cargar la pagina</a>
                    </p>
                </body>
        </html>
```

Cada vez que recargues la página o pulses el enlace, el contador aumentará. Esto sucede porque el valor de visitas se mantiene en la sesión entre peticiones.

Si destruyes la sesión y vuelves a entrar, el contador comenzará de nuevo cuando PHP cree o recupere una sesión sin esa variable.

7. EL IDENTIFICADOR DE SESIÓN Y LA CONFIGURACIÓN DE PHP

En el archivo **php.ini** existen diferentes directivas que permiten configurar el funcionamiento de las sesiones. Es importante conocerlas para entender cómo se identifica al usuario y dónde se guardan sus datos.

- **session.save_path**: Indica la ubicación de los archivos de sesión cuando se utiliza el almacenamiento en archivos.
- **session.use_cookies:** Controla si se utilizan cookies para transmitir el identificador de sesión.
- **session.name:** Define el nombre de la cookie de sesión. Su valor habitual es PHPSESSID.
- **session.auto_start:** Si está habilitada, permite iniciar automáticamente las sesiones.
- **session.cookie_lifetime:** Define la duración de la cookie de sesión en el navegador.
- **session.cookie_path:** Define las rutas para las que es válida la cookie.
- **session.use_trans_sid:** Controla el uso de mecanismos alternativos para transmitir el identificador de sesión, si están disponibles y habilitados.

A. QUÉ OCURRE CUANDO EJECUTAMOS SESSION_START() ?

La función no crea necesariamente una sesión nueva cada vez que se ejecuta.

- Si PHP recibe un identificador de sesión válido, intenta recuperar los datos asociados a él.
- Si no recibe un identificador válido, puede crear una nueva sesión.
- En ambos casos, la información queda disponible mediante **$_SESSION.**

Por eso, podemos incluir **session_start()** al principio de las distintas páginas PHP que necesiten acceder a la misma sesión.

B. ¿QUÉ OCURRE SI EL NAVEGADOR NO ACEPTA COOKIES?

Normalmente, PHP utiliza una cookie para transmitir el identificador de sesión. Si las cookies no están disponibles, existen mecanismos alternativos de propagación del identificador, dependiendo de la configuración de PHP.

Históricamente, **session.use_trans_sid** permitía transmitir el identificador mediante URL o campos ocultos de formularios. Sin embargo, transmitir identificadores de sesión de esa forma presenta riesgos de seguridad, por lo que no se recomienda habilitarlo como solución habitual.

En aplicaciones reales, es preferible utilizar cookies de sesión configuradas de forma segura.

8. ¿DÓNDE SE ALMACENA FÍSICAMENTE LA INFORMACIÓN?

Hasta ahora hemos trabajado con **$_SESSION** sin preocuparnos por el lugar exacto donde PHP guarda sus datos. No obstante, el servidor necesita almacenarlos de alguna forma para poder recuperarlos en peticiones posteriores.

La ubicación y el mecanismo dependen de la configuración del servidor.

A. ALMACENAMIENTO EN ARCHIVOS

En muchas instalaciones de PHP, el mecanismo predeterminado es files. Esto significa que PHP almacena los datos de sesión en archivos del servidor.

La directiva **session.save_path** indica la ubicación correspondiente.

Es importante que estos archivos estén protegidos y no sean accesibles directamente desde el navegador.

B. DIFERENCIA ENTRE NAVEGADOR Y SERVIDOR

Navegador

Guarda normalmente una cookie con el identificador de sesión.

PHPSESSID=identificador

El identificador permite asociar las peticiones con la sesión.

Servidor PHP

Almacena los datos de la sesión y permite recuperar sus valores.

$_SESSION["Total"] = 1550;

Esta separación es importante: el navegador no necesita conocer el contenido completo de la sesión, sino enviar el identificador que permite al servidor localizar los datos correspondientes.

C. LA CACHÉ DE PÁGINAS

En determinadas aplicaciones, los servidores intermedios o proxies pueden almacenar copias de páginas web para acelerar su entrega. Si se almacenan páginas personalizadas sin una configuración adecuada, existe el riesgo de mostrar información de un usuario a otro.

Las páginas que contienen información privada o personalizada deben gestionarse con las medidas de caché apropiadas. En PHP existen mecanismos como **session_cache_limiter()**, que debe configurarse antes de iniciar la sesión si se desea modificar el comportamiento predeterminado.

No hay que confundir este mecanismo con el almacenamiento de los datos de **$_SESSION:** una cosa es conservar los datos de la sesión y otra almacenar una copia de una página generada.